Ecuaciones diferenciales ordinarias II

Método de Runge–Kutta de cuarto orden y solución mediante `solve_ivp`

Computación Avanzada y Analítica

Objetivos de la clase

Al finalizar la clase, el estudiante podrá:

- Explicar la idea fundamental del método de Runge–Kutta de cuarto orden (RK4).
- Implementar RK4 para una EDO de primer orden.
- Analizar el error y la influencia del tamaño de paso.
- Resolver problemas de valor inicial utilizando `solve_ivp`.
- Interpretar los principales parámetros de `solve_ivp`.
- Diferenciar una implementación de RK4 de paso fijo de un método adaptativo como RK45.

Recordemos: problema de valor inicial

Consideremos una ecuación diferencial ordinaria de primer orden:

$$\frac{dy}{dt} = f(t, y), \qquad y(t_0) = y_0.$$

Queremos aproximar la solución en un intervalo

$$t_0 \leq t \leq t_f.$$

Dividimos el intervalo mediante

$$t_{n+1} = t_n + h,$$

donde h es el tamaño del paso.

Pregunta

¿Cómo podemos avanzar desde (t_n, y_n) hasta (t_{n+1}, y_{n+1})?

Repaso: método de Euler

El método de Euler utiliza una única evaluación de la pendiente:

$$\boxed{y_{n+1} = y_n + hf(t_n, y_n)}$$

La aproximación utiliza la pendiente al comienzo del intervalo.

Limitación

La pendiente puede cambiar significativamente dentro del intervalo $[t_n, t_n + h]$.

Idea

¿Por qué no evaluar la pendiente varias veces dentro del intervalo?

Idea de los métodos Runge–Kutta

Los métodos de Runge–Kutta utilizan varias evaluaciones de

$$f(t, y)$$

dentro de cada intervalo.

Para RK4 utilizaremos cuatro estimaciones:

$$\boxed{k_1, \quad k_2, \quad k_3, \quad k_4}$$

Las evaluaciones representan información sobre la pendiente:

- al comienzo;
- en el punto medio;
- nuevamente en el punto medio;
- al final del intervalo.

Primera pendiente: k_1

Partimos del punto conocido

$$(t_n, y_n).$$

La primera pendiente es

$$\boxed{k_1 = f(t_n, y_n)}$$

Esta es exactamente la pendiente de la solución en el punto inicial.

Podemos interpretarla como la información que utilizaría Euler para realizar su avance.

Segunda pendiente: k_2

Utilizamos k_1 para estimar el valor de la solución en el punto medio:

$$t_n + \frac{h}{2}.$$

La estimación de y es

$$y_n + \frac{h}{2}k_1.$$

Por tanto,

$$\boxed{k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right)}$$

Tercera pendiente: k_3

Realizamos nuevamente una evaluación en el punto medio.
Ahora utilizamos k_2:

$$\boxed{k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right)}$$

Tenemos ahora dos estimaciones independientes de la pendiente alrededor del centro del intervalo.

Cuarta pendiente: k_4

Finalmente avanzamos hasta el extremo derecho del intervalo:

$$t_n + h.$$

Utilizando k_3 estimamos:

$$y_n + hk_3.$$

Así,

$$\boxed{k_4 = f(t_n + h, y_n + hk_3)}$$

Fórmula completa de RK4

Las cuatro pendientes se combinan mediante

$$\boxed{y_{n+1} = y_n + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)}$$

Los pesos son

$$\boxed{1, \quad 2, \quad 2, \quad 1}$$

Las evaluaciones centrales reciben mayor peso.

Interpretación de RK4

Pendiente	Ubicación	Información
k_1	Inicio	Pendiente inicial
k_2	Punto medio	Primera estimación intermedia
k_3	Punto medio	Segunda estimación intermedia
k_4	Final	Pendiente final estimada

RK4 utiliza información de todo el intervalo para construir una única pendiente promedio ponderada.

Algoritmo RK4

Para cada paso:

1. Calcular k_1.
2. Calcular k_2 utilizando k_1.
3. Calcular k_3 utilizando k_2.
4. Calcular k_4 utilizando k_3.
5. Combinar las cuatro pendientes.
6. Repetir.

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

Orden y error

RK4 es un método de cuarto orden.
Su error local de truncamiento es

$$\boxed{O(h^5)}$$

mientras que el error global es

$$\boxed{O(h^4)}.$$

Si reducimos el paso aproximadamente a la mitad:

$$E(h/2) \approx \frac{E(h)}{2^4} = \frac{E(h)}{16}.$$

Importante

Esta relación es aproximada y corresponde al régimen donde domina el error de discretización.

Implementación de RK4

```
def rk4(f, t0, tf, y0, h):
    t = np.arange(t0, tf + h, h)
    y = np.zeros(len(t))
    y[0] = y0

    for n in range(len(t) - 1):
        k1 = f(t[n], y[n])
        k2 = f(
            t[n] + h/2,
            y[n] + h*k1/2
        )
        k3 = f(
            t[n] + h/2,
            y[n] + h*k2/2
        )
        k4 = f(
            t[n] + h,
            y[n] + h*k3
        )
        y[n+1] = y[n] + h/6 * (
            k1 + 2*k2 + 2*k3 + k4
        )
    return t, y
```

Ejercicio resuelto 1

Consideremos:

$$\frac{dy}{dt} = -2y, \qquad y(0) = 1.$$

La solución exacta es

$$\boxed{y(t) = e^{-2t}}$$

Utilizaremos esta solución para comprobar nuestra implementación numérica.

Ejercicio 1: definición de la EDO

```
def f(t, y):
    return -2*y
```

Aplicamos RK4:

```
t, y_rk4 = rk4(
    f,
    0,
    5,
    1,
    0.1
)
```

Ejercicio 1: solución exacta y error

```
y_exacta = np.exp(-2*t)

error = np.abs(
    y_exacta - y_rk4
)

print(
    "Error maximo:",
    np.max(error)
)
```

Pregunta

¿Qué esperamos observar al comparar la solución exacta y la solución de RK4?

Ejercicio 1: gráfica

```
plt.plot(
    t, y_exacta,
    label="Solucion exacta"
)

plt.plot(
    t, y_rk4,
    "o",
    markersize=3,
    label="RK4"
)

plt.xlabel("t")
plt.ylabel("y(t)")
plt.grid()
plt.legend()
plt.show()
```

Experimento: efecto de h

Probemos diferentes tamaños de paso:

$$h = 0.5, \qquad h = 0.1, \qquad h = 0.05, \qquad h = 0.01.$$

Para cada caso calculamos:

$$E_{\text{máx}} = \max_{n} \left| y_{\text{exacta}}(t_n) - y_{\text{RK4}}(t_n) \right|.$$

Pregunta

¿El error disminuye aproximadamente como h^4?

De RK4 a SciPy

Ya sabemos implementar RK4.
Ahora queremos utilizar una herramienta científica que permita resolver problemas de valor inicial de manera más general.

`scipy.integrate.solve_ivp`

- Reduce la cantidad de código que debemos escribir.
- Permite diferentes métodos de integración.
- Permite control adaptativo del tamaño de paso.
- Puede resolver sistemas de ecuaciones diferenciales.

Estructura básica de `solve_ivp`

Primero importamos:

```
from scipy.integrate import solve_ivp
```

La estructura básica es:

```
sol = solve_ivp(
    fun,
    t_span,
    y0
)
```

Principales argumentos

`fun`	función que representa $f(t, y)$.
`t_span`	intervalo (t_0, t_f).
`y0`	condición inicial.
`t_eval`	tiempos donde queremos la solución.
`method`	método numérico.
`rtol`	tolerancia relativa.
`atol`	tolerancia absoluta.

Ejemplo con solve_ivp

Para

$$y' = -2y, \qquad y(0) = 1,$$

podemos escribir:

```
def f(t, y):
    return -2*y

t_eval = np.linspace(0, 5, 100)

sol = solve_ivp(
    f,
    (0, 5),
    [1],
    t_eval=t_eval,
    method="RK45"
)
```

¿Qué contiene `sol`?

Algunos elementos importantes son:

- `sol.t`: tiempos calculados.
- `sol.y`: valores de la solución.
- `sol.success`: indica si la integración terminó correctamente.
- `sol.message`: información sobre el resultado.

Para una sola variable:

```
sol.y[0]
```

contiene la solución numérica.

Control adaptativo del paso

El metodo RK45 de `solve_ivp` utiliza una estimacion del error.
El algoritmo puede cambiar automaticamente el tamano del paso.
Si el error estimado es grande, puede reducir el paso.
Si el error estimado es pequeno, puede aumentar el paso.

RK4 manual vs. RK45

Caracteristica	RK4 manual	RK45 en `solve_ivp`
Paso	Fijo	Adaptativo
Codigo	Implementado por nosotros	Biblioteca
Control del paso	No	Si
Objetivo	Comprension del algoritmo	Resolucion general

Tolerancias

Dos parametros importantes son:

$$\texttt{rtol} \quad \text{y} \quad \texttt{atol}.$$

`rtol`: tolerancia relativa.
`atol`: tolerancia absoluta.
Conceptualmente, el control del error utiliza una escala relacionada con

$$\texttt{atol} + \texttt{rtol}|y|.$$

Las tolerancias influyen en el tamano de los pasos internos.

Ejercicio resuelto 2: caída con resistencia

Consideremos:

$$m\frac{dv}{dt} = mg - cv.$$

Despejando:

$$\boxed{\frac{dv}{dt} = g - \frac{c}{m}v}$$

Usaremos:

$$m = 2\,\mathrm{kg}, \qquad c = 0.5\,\mathrm{kg/s}, \qquad g = 9.81\,\mathrm{m/s^2}.$$

Condición inicial:

$$v(0) = 0.$$

Velocidad terminal

En la velocidad terminal:

$$\frac{dv}{dt} = 0.$$

Por tanto,

$$0 = g - \frac{c}{m} v_T$$

y

$$\boxed{v_T = \frac{mg}{c}}$$

Para nuestros parámetros:

$$\boxed{v_T = 39.24\,\mathrm{m/s}}$$

Ejercicio 2: definición del modelo

```
m = 2.0
c = 0.5
g = 9.81

def dvdt(t, v):
    return g - (c/m)*v

t_eval = np.linspace(0, 20, 200)
```

Ejercicio 2: solve_ivp

```
sol = solve_ivp(
    dvdt,
    (0, 20),
    [0],
    t_eval=t_eval,
    method="RK45"
)
```

La solución está disponible en:

```
sol.t
sol.y[0]
```

Ejercicio 2: visualización

```
plt.plot(
    sol.t,
    sol.y[0],
    label="v(t)"
)

plt.axhline(
    m*g/c,
    linestyle="--",
    label="Velocidad terminal"
)

plt.xlabel("Tiempo [s]")
plt.ylabel("Velocidad [m/s]")
plt.grid()
plt.legend()
plt.show()
```

Interpretación física

Al inicio:

$$v(0) = 0.$$

Por tanto, la resistencia es inicialmente cero y

$$\frac{dv}{dt} = g.$$

A medida que aumenta v, aumenta la resistencia cv.
Finalmente:

$$mg - cv \to 0.$$

La aceleración se aproxima a cero y la velocidad se aproxima a

$$v_T = \frac{mg}{c}.$$

¿Qué ocurre si cambiamos los parámetros?

La velocidad terminal es

$$v_T = \frac{mg}{c}.$$

Por tanto:

- Si aumenta g, aumenta v_T.
- Si aumenta m, aumenta v_T.
- Si aumenta c, disminuye v_T.

Además, los parámetros modifican la rapidez con la que se alcanza la región cercana a la velocidad terminal.

Sistemas de EDO

Muchas ecuaciones físicas son de segundo orden.
Por ejemplo:

$$\frac{d^2x}{dt^2} + \omega^2 x = 0.$$

Definimos:

$$v = \frac{dx}{dt}.$$

Entonces:

$$\frac{dx}{dt} = v$$

y

$$\frac{dv}{dt} = -\omega^2 x.$$

Obtenemos un sistema de dos EDO de primer orden.

Forma vectorial

Definimos el vector de estado:

$$\mathrm{y} = \begin{pmatrix} x \\ v \end{pmatrix}.$$

Entonces:

$$\boxed{\frac{d\mathrm{y}}{dt} = \mathrm{f}(t, \mathrm{y})}$$

con

$$\mathrm{f}(t, \mathrm{y}) = \begin{pmatrix} v \\ -\omega^2 x \end{pmatrix}.$$

Esta es la forma que recibe `solve_ivp`.

Oscilador armónico

Para

$$\omega = 2, \qquad x(0) = 1, \qquad v(0) = 0,$$

podemos escribir:

```
omega = 2.0

def oscilador(t, y):
    x, v = y

    dxdt = v
    dvdt = -(omega**2)*x

    return [dxdt, dvdt]
```

Resolver el sistema

```
t_eval = np.linspace(0, 10, 500)

sol = solve_ivp(
    oscilador,
    (0, 10),
    [1, 0],
    t_eval=t_eval,
    method="RK45"
)

x = sol.y[0]
v = sol.y[1]
```

Solución analítica

Para las condiciones

$$x(0) = 1, \qquad v(0) = 0,$$

la solución exacta de la posición es

$$\boxed{x(t) = \cos(\omega t)}$$

Podemos comparar:

$$x_{\text{num}}(t)$$

con

$$x_{\text{exacta}}(t).$$

Ejercicios para casa

Ejercicio 1: ecuación logística

$$\frac{dy}{dt} = y(1 - y), \qquad y(0) = 0.1.$$

Resolver en

$$0 \leq t \leq 10.$$

Implementar RK4, resolver con `solve_ivp`, graficar y comparar los resultados.

Ejercicio 2: oscilador armónico

$$x'' + \omega^2 x = 0,$$

con

$$x(0) = 1, \qquad v(0) = 0, \qquad \omega = 2.$$

Transformar a sistema y resolver con `solve_ivp`.

Preguntas para cerrar

1. ¿Por qué RK4 utiliza cuatro pendientes?
2. ¿Qué representa cada k_i?
3. ¿Qué significa que RK4 sea de cuarto orden?
4. ¿Qué diferencia existe entre RK4 y RK45?
5. ¿Para qué sirve `t_eval`?
6. ¿Qué función cumplen `rtol` y `atol`?
7. ¿Cómo convertimos una EDO de segundo orden en un sistema de primer orden?

Ideas clave

- RK4 combina cuatro estimaciones de la pendiente.
- Su error global es de orden $O(h^4)$.
- El tamaño de paso afecta directamente la precisión.
- `solve_ivp` permite resolver problemas de valor inicial de forma general.
- RK45 utiliza control adaptativo del tamaño de paso.
- Los sistemas de EDO permiten modelar problemas físicos más complejos.

Una solución numérica debe calcularse, verificarse e interpretarse.